



University of Asia Pacific

DEPARTMENT OF CSE

COURSE CODE : 402

COURSE TITLE : Operating System Lab

DATE OF SUBMISSION : 29/08/2026

SUBMITTED BY:

NAME : Fairoj Salim

STUDENT ID : 23101165

SECTION : C2

LEVEL : 4th Year 1st Semester

SUBMITTED TO:

Atia Rahman Orthi
LECTURER
Department of CSE, UAP

PROBLEM STATEMENT

Implement Preemptive SJF (Shortest Job First, also known as SRTF - Shortest Remaining Time First) CPU Scheduling for the following scenario where the number of processes = 5 and the process information is:

P_id	AT	BT
p1	4	2
p2	2	2
p3	1	3
p4	0	6
p5	3	1

Calculate CT, TAT, WT, Average TAT and Average WT using a Time Quantum = 2. Also represent the process execution sequence.

SOURCE CODE

```
p_id = ["p1", "p2", "p3", "p4", "p5"]
at = [4, 2, 1, 0, 3]
bt = [2, 2, 3, 6, 1]

n = len(p_id)

quantum = 2

remaining = bt.copy()

ct = [0]*n
tat = [0]*n
wt = [0]*n

time = 0
completed = 0

execution_seq = []

while completed < n:
    shortest = -1

    while completed < n:
        shortest = -1

        for i in range(n):
            if at[i] <= time and remaining[i] > 0:
                if shortest == -1 or remaining[i] < remaining[shortest]:
```

```

        shortest = i

    if shortest == -1:

        time = time + 1

    else:

        execution_seq.append(p_id[shortest])

        if remaining[shortest] > quantum:

            time = time + quantum
            remaining[shortest] = remaining[shortest] - quantum

        else:

            time = time + remaining[shortest]
            remaining[shortest] = 0

        if remaining[shortest] == 0:
            ct[shortest] = time
            completed = completed + 1

sum_tat = 0
sum_wt = 0

for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]

    sum_tat = sum_tat + tat[i]
    sum_wt = sum_wt + wt[i]

avg_tat = sum_tat / n
avg_wt = sum_wt / n

print("\nPREEMPTIVE SJF:")

print("Time Quantum =", quantum)

print("Execution Sequence:", end=" ")

for process in execution_seq:
    print(process, end=" ")

print("\n")

print(f'{"PID":<8}{"AT":<6}{"BT":<6}{"CT":<6}{"TAT":<6}{"WT":<6}')
```

```

for i in range(n):

    print(
        f'{"p_id[i]":<8}{"at[i]":<6}{"bt[i]":<6}'
        f'{"ct[i]":<6}{"tat[i]":<6}{"wt[i]":<6}'
    )

print("\nAverage TAT =", avg_tat)
print("Average WT =", avg_wt)

```

OUTPUT

```
C:\Users\lab\PycharmProjects\PythonProject\.venv\Scripts\python.exe "C:\Users\lab\PycharmProjects\PythonProject\main.py"

PREEMPTIVE SJF:
Time Quantum = 2
Execution Sequence: p4 p2 p5 p1 p3 p3 p4 p4

PID    AT    BT    CT    TAT   WT
p1      4     2     7     3     1
p2      2     2     4     2     0
p3      1     3    10     9     6
p4      0     6    14    14     8
p5      3     1     5     2     1

Average TAT = 6.0
Average WT  = 3.2

Process finished with exit code 0
```

DISCUSSION

In this experiment, Preemptive SJF (also called SRTF - Shortest Remaining Time First) CPU scheduling was implemented in Python and its performance was analyzed using Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT).

Unlike Non-Preemptive SJF, where once a process is selected it runs to completion without interruption, Preemptive SJF re-evaluates the ready queue continuously and can interrupt a running process if a process with a shorter remaining burst time becomes available. To implement this, I checked at every time step which processes had already arrived and still had remaining burst time, and among them selected the one with the smallest remaining burst time. This selected process was then allowed to run for at most one Time Quantum before the CPU re-checked whether a shorter job had arrived in the meantime, which is what causes a single process to appear multiple times in the execution sequence.

From the output, the execution sequence obtained was $p4 \rightarrow p2 \rightarrow p5 \rightarrow p1 \rightarrow p3 \rightarrow p3 \rightarrow p4 \rightarrow p4$, which shows p3 and p4 being interrupted and resumed multiple times as shorter processes (p2, p5, p1) arrived and were prioritized ahead of them. The average Turnaround Time was 6.0 and the average Waiting Time was 3.2.

Comparison: Non-Preemptive SJF vs Preemptive SJF

- **Execution:** Non-Preemptive SJF runs the selected process to completion; Preemptive SJF (SRTF) can interrupt it if a shorter process arrives.
- **Decision point:** Non-Preemptive SJF picks a process only when the CPU is free; Preemptive SJF re-checks the ready queue continuously.

- **Waiting Time:** Preemptive SJF gives lower average WT and TAT, since short processes don't have to wait behind a long one already running.
- **Overhead:** Non-Preemptive SJF has less context-switching overhead; Preemptive SJF has more, due to frequent interruptions.
- **Fairness to long processes:** Non-Preemptive SJF is fairer to long processes; Preemptive SJF can starve them if short processes keep arriving.
- **Simplicity:** Non-Preemptive SJF is simpler to implement; Preemptive SJF is more complex due to constant re-evaluation.
- **In this process set:** p4 (BT = 6) got interrupted by shorter arrivals (p2, p5, p1), which is why Preemptive SJF produced a lower Average TAT (6.0) and WT (3.2) than Non-Preemptive SJF would.

CONCLUSION

This experiment provided practical experience in implementing Preemptive SJF (SRTF) CPU scheduling using Python and comparing its behavior with Non-Preemptive SJF. Implementing the preemptive version required continuously re-checking the ready queue at every quantum interval instead of committing to a single process until completion, which made the scheduling logic more complex than the non-preemptive case but allowed the algorithm to react immediately to newly arrived shorter processes.

The results show that for this process set, the execution produced an average Turnaround Time of 6.0 and an average Waiting Time of 3.2, with processes p3 and p4 being preempted and resumed as shorter jobs (p2, p5, p1) arrived and were prioritized ahead of them.

Preemptive SJF vs Non-Preemptive SJF: The Better Choice

- **Overall winner:** Preemptive SJF (SRTF) is better than Non-Preemptive SJF for this process set.
- **Why:** It reacts immediately to shorter arriving processes instead of blocking the CPU with a long one already running.
- **Result:** This gives a lower average Waiting Time (3.2) and Turnaround Time (6.0) than Non-Preemptive SJF would produce here.
- **Trade-off:** Preemptive SJF adds context-switching overhead and can starve long processes, but for minimizing average TAT/WT, it still comes out ahead.